

# LLM Research Papers: The 2026 List (January to May)



SEBASTIAN RASCHKA, PHD

JUN 06, 2026 · PAID



86



3



11

Share

## LLM Research Papers: The 2026 List (January to May)

As some of you know, I have the long-running habit of keeping a running list of research papers I want to read, revisit, or cite in future articles and projects.

Last year, I shared two organized paper lists, [one](#) covering January to June and [another](#) covering July to December.

Several readers told me that these lists were very useful, so, in a similar spirit, I prepared a new list for the first half of 2026. This one covers papers I bookmarked from January through May 2026.

Please do not treat this as a complete list of everything published this year. There are so many papers published every day that this would be totally infeasible. Instead, this is a curated reference list based on papers I found interesting or relevant for my own work. I went through the titles, abstracts, and topic framing carefully while organizing the list, but I have to admit that I also only read a subset of the papers in detail.

Why make these lists in the first place? When I work on an article, book section, conference example, or lecture, I often remember that I saw a relevant paper somewhere, but finding it again can be surprisingly annoying. A categorized Markdown list solves this problem for me, and I hope it is useful to you as well. (Even in the era of LLM-based

web searching, having a specific context list is pretty useful, still.)

This year, the list is again heavy on reasoning models, reinforcement learning, and efficient inference, because I am biased towards bookmarking papers that are related to things I am currently working on. However, compared with the 2025 lists, I also bookmarked more papers around agent harnesses, tool use, long context, diffusion language models, and practical serving infrastructure, because that's what I am currently pretty involved in and where the field is headed.

The categories for this research paper list are as follows. (Pro tip: In the web version of this article, you can use the table of contents on the left to jump directly to the sections that are most relevant to you.)

1. Architecture and Model Design
2. Efficient Training and Scaling
3. Inference Efficiency and KV Cache
4. Sparse Attention and Long Context
5. Reasoning and Test-Time Compute
6. Reinforcement Learning and RLVR
7. Agent Systems and Tool Use
8. Coding Agents and Software Engineering
9. Diffusion Language Models
10. Model Evaluation and Benchmarks

## 1. Architecture and Model Design

This first section collects papers on model architecture, model-release technical reports, and papers that help explain why current LLMs look the way they do.

One thing I find interesting about 2026 so far is that architecture work goes beyond making transformers larger. There is a lot of work around

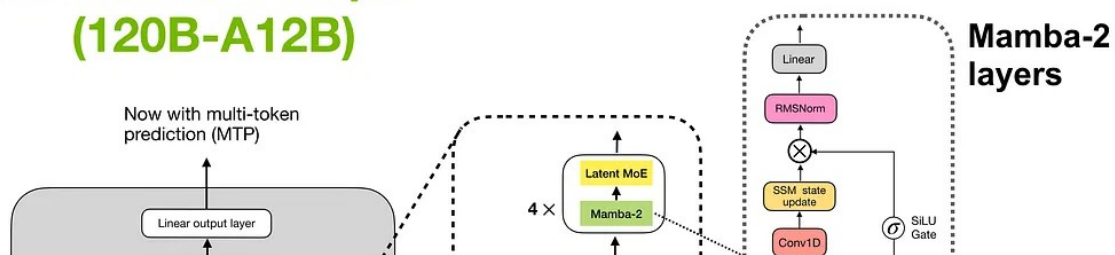
- hybrid architectures (for example, [Nemotron 3](#), and [Arcee Trinity](#)),
- state space layers ([Nemotron 3](#) and [Mamba-3](#)),
- MoE capacity allocation ([Scaling Embeddings Outperforms Scaling Experts](#), and [3.5 Flash](#)),
- activation behavior ([The Spike, the Sparse and the Sink](#)),
- and representation geometry ([Symmetry in Language Statistics Shapes the Geometry of Model Representations](#)).

All of these papers are quite interesting, which is why I bookmarked them in the first place. But if I had to pick one must-read, I'd probably be Nemotron 3 Super, because the article is *super* detailed (no pun intended), and it describes techniques used in a model that is already in production. And it's one of the best models in its size class after all.

One of the interesting aspects of Nemotron 3 is its hybrid-architecture design, meaning that it alternates between regular attention layers and Mamba-2 (state space model) layers to be more efficient at long contexts. In 2026, long-context efficiency is becoming more and more important as more and more LLMs get plugged into agent harnesses (OpenClaw etc.), which requires working with longer and longer contexts.

That being said, 120B-A12B may be a bit too large for local inference on regular consumer hardware, but there is a Nemotron 3 Nano (4B) version as well.

## Nemotron 3 Super (120B-A12B)



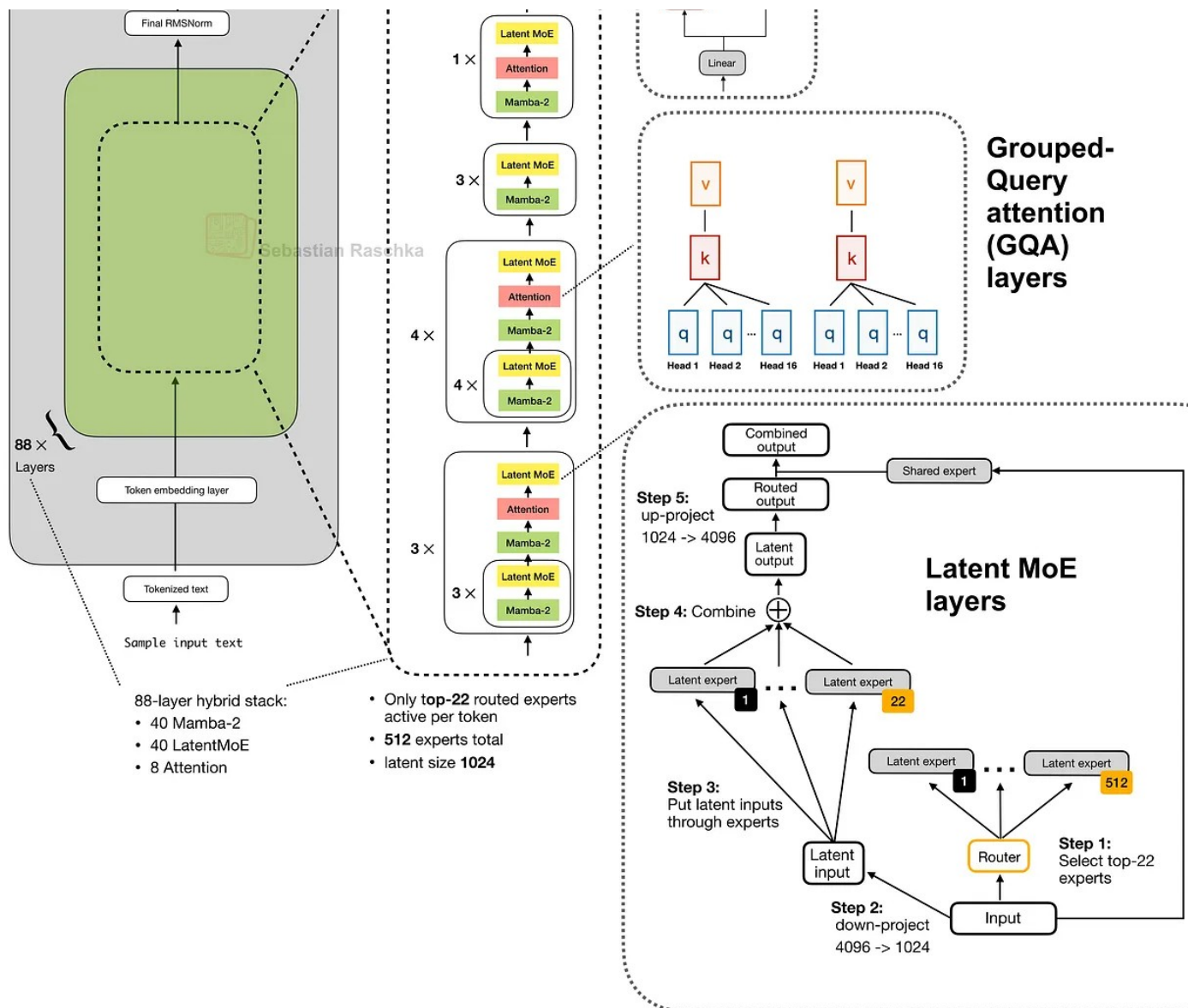


Figure 1: Architecture of Nemotron-3 Super, which is a hybrid architecture using Mamba-2 layers.

Note that 2 days ago, Nvidia also released a scaled up-version of this, Nemotron 3 Ultra (550B-A55B), which scales the embedding and projection dimensions but otherwise uses the same building blocks. If you are interested in a visual, I posted about it on Substack Notes [here](#).

This hybrid-architecture trend with alternating attention and alternative layers is a relatively popular development this year. The probably most popular open-weight L series that uses a similar hybrid design is probably Qwen3.6, which uses Gated DeltaNet layers instead of Mamba-2 layers for the non-attention portions. For more

information, see my Hybrid Attention (<https://sebastianraschka.com/llm-architecture-gallery/hybrid-attention/>) write-up, which pools information from several of my previous substack articles where I wrote about these.

Also, in the paper list below, you may notice that there is now a Mamba-3 and Gate DeltaNet-2 (i.e., newer versions of Mamba-2 and GatedDeltaNet), and it will be interesting to see those in the upcoming open-weight LLMs (e.g., Nemotron-4 and Qwen4?).

Next to describing the hybrid-architecture design, the Nemotron-3 paper contains a whole lot of other interesting ablations, for example, around multi-token prediction, speculative decoding, NVFP4 pretraining versus BF16, synthetic MMLU-style data, a post-training quantization recipes, but covering these in detail would be out of scope for this overview.

- 1 Jan, Deep Delta Learning, <https://arxiv.org/abs/2601.00417>
- 6 Jan, MiMo-V2-Flash Technical Report, <https://arxiv.org/abs/2601.02780>
- 13 Jan, Ministral 3, <https://arxiv.org/abs/2601.08584>
- 29 Jan, Scaling Embeddings Outperforms Scaling Experts in Language Models, <https://arxiv.org/abs/2601.21204>
- 30 Jan, LatentLens: Revealing Highly Interpretable Visual Tokens in LLMs, <https://arxiv.org/abs/2602.00462>
- 4 Feb, ERNIE 5.0 Technical Report, <https://arxiv.org/abs/2602.04705>
- 8 Feb, ViT-5: Vision Transformers for the Mid-2020s, <https://arxiv.org/abs/2602.08071> (Most of this article is LLM-focused, but I couldn't resist to include a new major vision transformer design.)
- 11 Feb, Step 3.5 Flash: Open Frontier-Level Intelligence with 11B Active Parameters, <https://arxiv.org/abs/2602.10604>
- 12 Feb, Nanbeige4.1-3B: A Small General Model That Reasons, Aligns, and Acts

<https://arxiv.org/abs/2602.13367>

- 16 Feb, Symmetry in Language Statistics Shapes the Geometry of Model Representations, <https://arxiv.org/abs/2602.15029>
- 17 Feb, GLM-5: From Vibe Coding to Agentic Engineering, <https://arxiv.org/abs/2602.15763>
- 18 Feb, Arcee Trinity Large Technical Report, <https://www.arxiv.org/abs/2602.17>
- 4 Mar, The Spike, the Sparse and the Sink: Anatomy of Massive Activations and Attention Sinks, <https://arxiv.org/abs/2603.05498>
- 12 Mar, Tiny Aya: Bridging Scale and Multilingual Depth, <https://arxiv.org/abs/2603.11510>
- 15 Mar, Attention Residuals, <https://arxiv.org/abs/2603.15031>
- 16 Mar, Mamba-3: Improved Sequence Modeling Using State Space Principles, <https://arxiv.org/abs/2603.15569>
- 31 Mar, Attention to Mamba: A Recipe for Cross-Architecture Distillation, <https://arxiv.org/abs/2604.14191>
- 13 Apr, Nemotron 3 Super: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning, <https://arxiv.org/abs/2604.12374>
- 6 May, ZAYA1-8B Technical Report, <https://arxiv.org/abs/2605.05365>
- 13 May, Delta Attention Residuals, <https://arxiv.org/abs/2605.18855>
- 21 May, Gated DeltaNet-2: Decoupling Erase and Write in Linear Attention, <https://arxiv.org/abs/2605.22791>
- 25 May, The MiniMax-M2 Series: Mini Activations Unleashing Max Real-World Intelligence, <https://arxiv.org/abs/2605.26494>

## 2. Efficient Training and Scaling

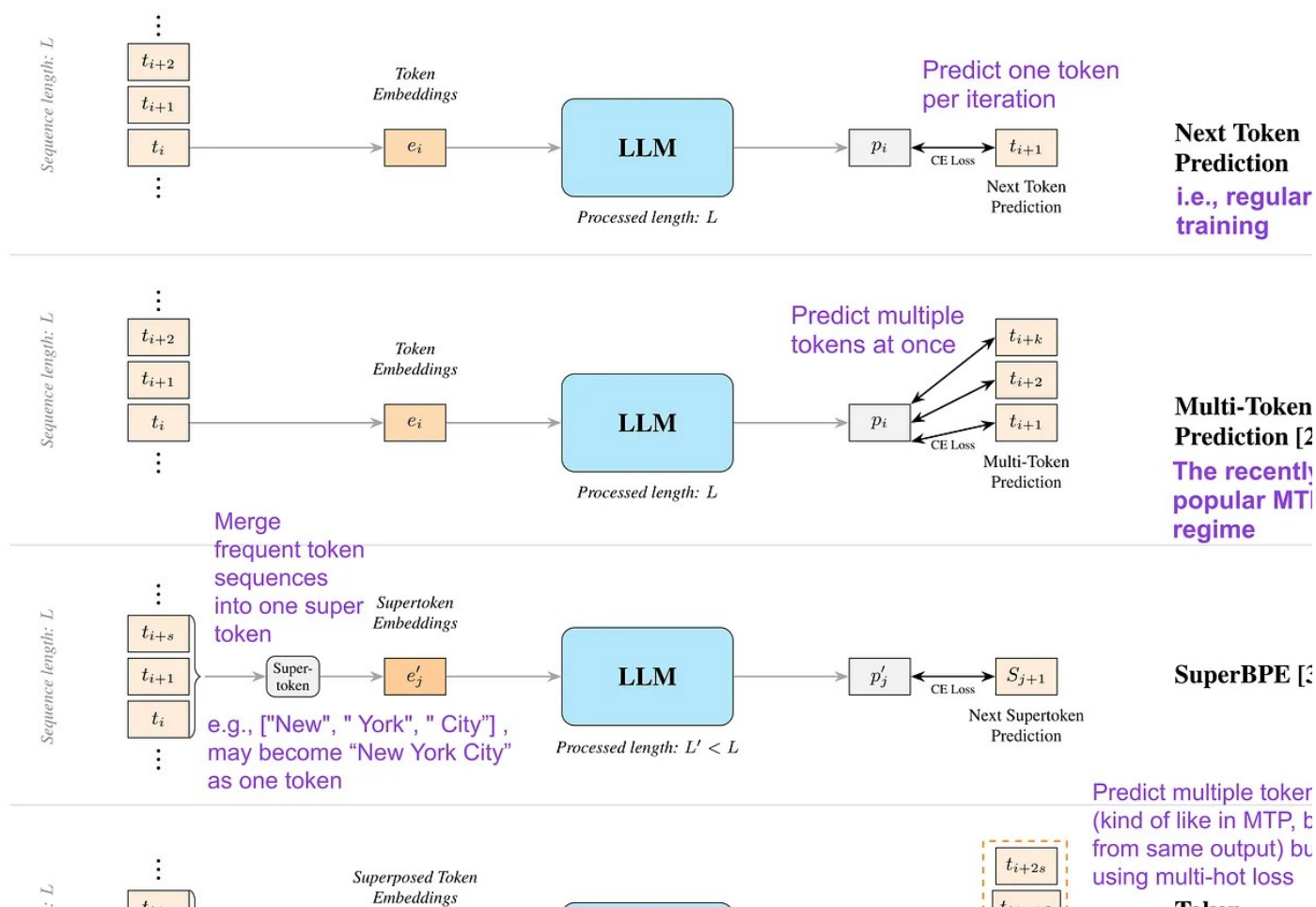
This section is about training systems, adaptation methods, and scaling recipes. The

papers are not (all) about pre-training from scratch. Some focus on fine-tuning, distillation, test-time training, or making training work better on constrained hardware.

I like keeping this bucket separate from architecture papers because many of the ideas here are more about the process than the model shape.

Like mentioned before, all of these papers are interesting, which is why I bookmarked them in the first place. But if I had to pick one *super* interesting must-read (again, no pun intended), I would probably pick [Efficient Pre-Training with Token Superposition](#).

The overall big-picture idea is to make part of pre-training coarser. Instead of always feeding the model one token at a time, Token Superposition Training (TST) averages contiguous token embeddings into a single latent token. The model then predicts the next set of tokens with a multi-hot objective. After this first phase, they then remove the TST code and continue with regular next-token prediction.



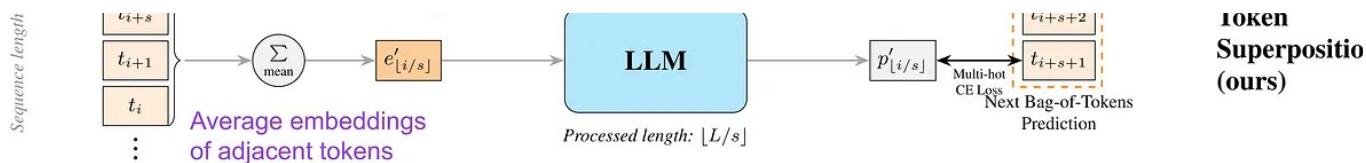


Figure 2: Conceptual overview of Token Superposition Training using an annotated figure from <https://arxiv.org/abs/2605.06546>.

What I like about this setup is that it does not require changing the model architecture, tokenizer, optimizer, data, etc. Also, TST is a training-time recipe, not something that remains attached to the deployed model.

The results are also quite convincing where across multiple runs, TST reaches lower loss than standard training under the same FLOPs budget.

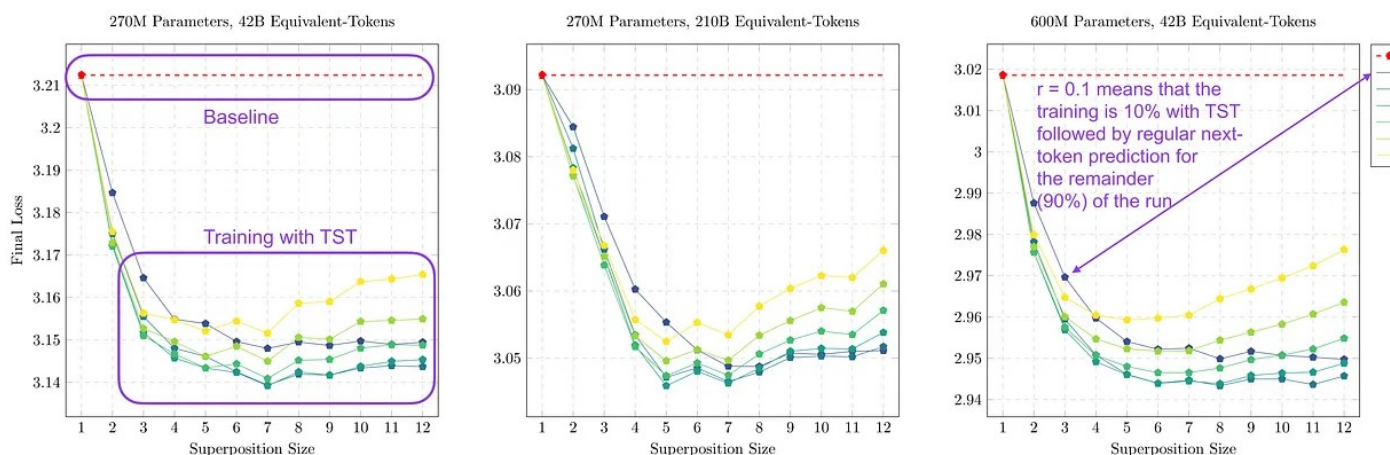


Figure 3: Annotated version of the paper's (<https://arxiv.org/abs/2605.06546>) training-loss results. The dashed red line is the standard next-token baseline. Several Token Superposition Training settings reach lower loss at the same FLOPs budget, and the same pattern appears in both the 270M and 600M parameter settings.

There are some caveats, though. The paper validates the recipe up to a 10B-A1B M model, which is useful, but still far from frontier-scale pre-training.

Still, I think this is a relatively interesting method.

- 27 Jan, Self-Distillation Enables Continual Learning, <https://arxiv.org/abs/2601.19897>

- 29 Jan, Shaping Capabilities with Token-Level Data Filtering, <https://arxiv.org/abs/2601.21571>
- 29 Jan, Self-Improving Pretraining: Using Post-Trained Models to Pretrain Better Models, <https://arxiv.org/abs/2601.21343>
- 31 Jan, BLOCK-EM: Preventing Emergent Misalignment via Latent Blocking, <https://arxiv.org/abs/2602.00767>
- 5 Feb, OPUS: Towards Efficient and Principled Data Selection in Large Language Model Pre-training in Every Iteration, <https://arxiv.org/abs/2602.05400>
- 24 Feb, Test-Time Training with KV Binding Is Secretly Linear Attention, <https://arxiv.org/abs/2602.21204>
- 5 Mar, Progressive Residual Warmup for Language Model Pretraining, <https://arxiv.org/abs/2603.05369>
- 12 Mar, Neural Thickets: Diverse Task Experts Are Dense Around Pretrained Weights, <https://arxiv.org/abs/2603.12228>
- 2 Apr, MiCA Learns More Knowledge Than LoRA and Full Fine-Tuning, <https://arxiv.org/abs/2604.01694>
- 7 Apr, In-Place Test-Time Training, <https://arxiv.org/abs/2604.06169>
- 22 Apr, Hybrid Policy Distillation for LLMs, <https://arxiv.org/abs/2604.20244>
- 27 Apr, Large Language Models Explore by Latent Distilling, <https://arxiv.org/abs/2604.24927>
- 29 Apr, Efficient Training on Multiple Consumer GPUs with RoundPipe, <https://arxiv.org/abs/2604.27085>
- 7 May, Efficient Pre-Training with Token Superposition, <https://arxiv.org/abs/2605.06546>
- 13 May, MinT: Managed Infrastructure for Training and Serving Millions of LLMs, <https://arxiv.org/abs/2605.13779>

(Shameless plug: Sten Ruediger and I worked on the [MiCA paper](#) a few years ago but finally got around to writing it up as a preprint; but Sten really did most of the work here!)

### 3. Inference Efficiency and KV Cache

Inference efficiency remains one of the most important practical topics for LLMs. For example, more efficient inference saves big LLM providers tons of money, lets individual users run larger models faster on consumer hardware, and is especially important in long-context (e.g., agent) use cases.

The papers below cover KV-cache reuse, pruning, efficient attention kernels, decoding methods, and memory-aware serving.

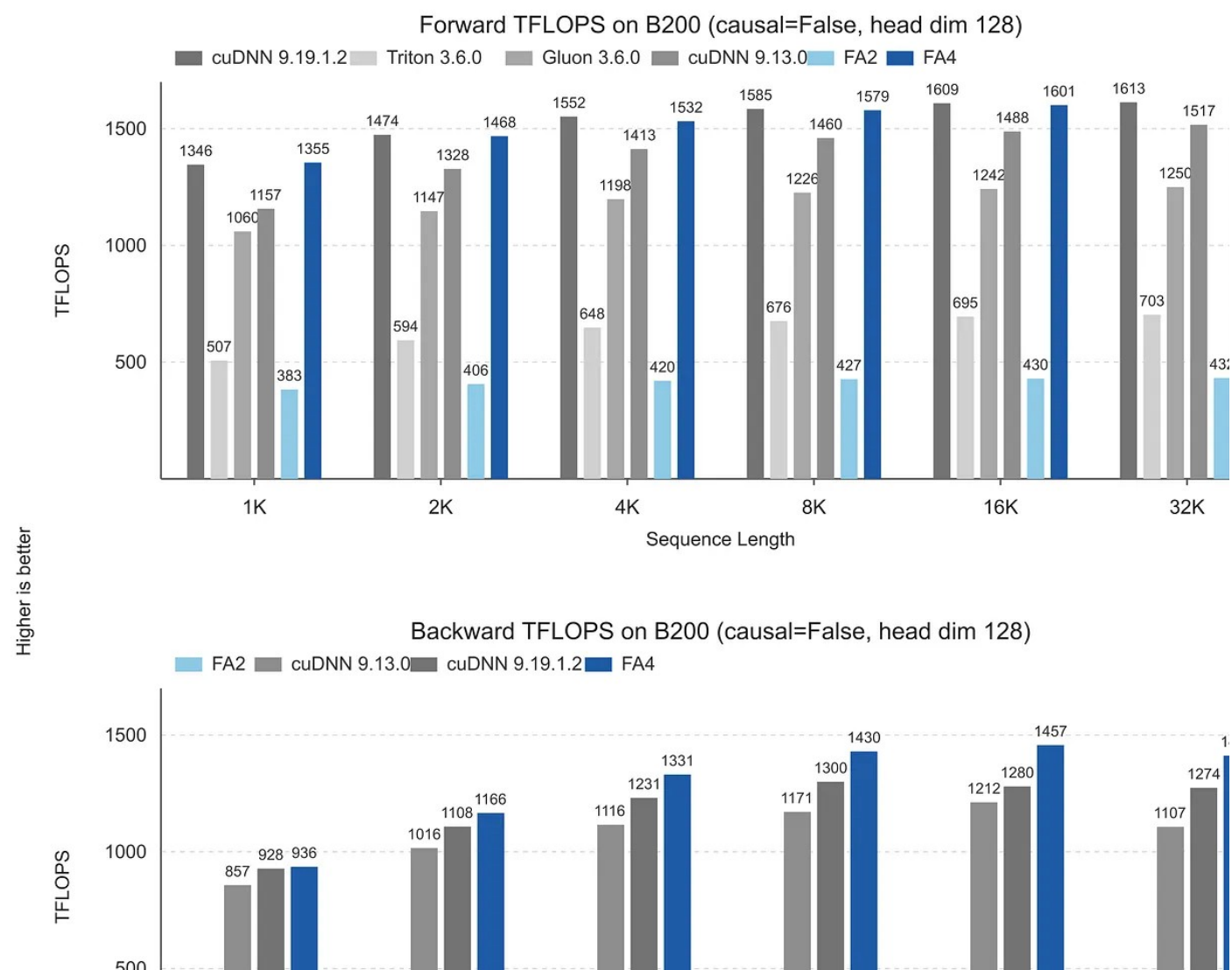
Here, I would pick [FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling](#), which is the successor of the popular FlashAttention implementations.

Side note: FlashAttention is an implementation-level improvement that computes the same attention result as standard attention. The trick is that it reorganizes the computation to avoid slow memory movement and use the GPU more efficiently. However, mathematically, the model still uses the same attention operation, so the output should be equivalent up to small numerical differences from floating-point arithmetic.

Back to the paper: FlashAttention-4 is interesting because attention is still one of the core bottlenecks for long-context LLMs, and FlashAttention-style kernels have a wide-ranging effect through downstream libraries. Similar to FlashAttention 1 to 3, I expect FlashAttention 4 to be widely used inside all kinds of libraries, from PyTorch and Hugging Face transformers to vLLM, SGLang, and other high-throughput serving stacks.

So, what's new in FlashAttention-4? The main point of the paper is that the more recent Blackwell GPUs changed the bottleneck. On B200 and GB200, tensor core throughput increased a lot compared to other parts of the chip. (Shared memory bandwidth and exponential units become relatively more limiting.) So, attention kernels now need to use tensor cores well while keeping the non-matrix-multiplication parts from slowing down the whole kernel.

FlashAttention-4 addresses this with a set of kernel and algorithm changes. For example, the paper describes several low-level GPU optimizations, including larger computation blocks, asynchronous matrix multiplications, faster approximate exponentials for softmax, better memory usage, and a backward pass that splits work across two cooperating CUDA thread blocks.



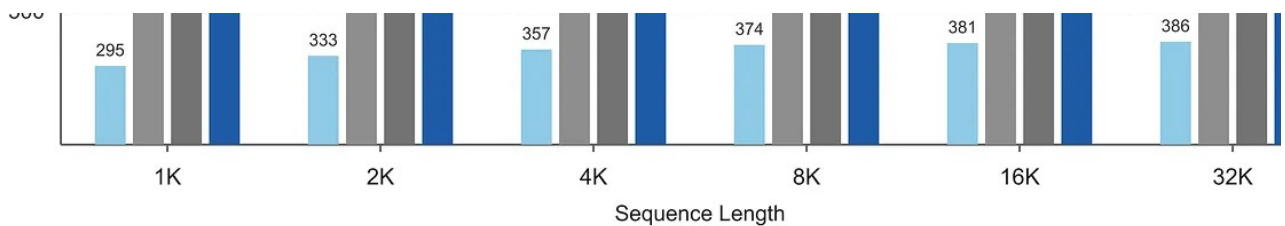


Figure 4: Forward and backward attention throughput on a B200 GPU from the FlashAttention-4 paper (<https://arxiv.org/abs/2603.05451>). I replotted this to highlight the differences between FA2 and FA4. The steps between 1K and 32K sequence lengths are omitted for clarity.

Note that they only compare it to FlashAttention-2 (FA2) not FlashAttention-3 (FA3) because FA3 was designed for Hopper GPUs (e.g., H100), and apparently does not run on B200 in their benchmark setup.

Back to the figure. As we can see, the speed-up of FA4 over FA2 is really substantial (about 4x). Also, it performs similar to the latest optimized attention kernels in cuDNN in the forward pass and performs substantially better in the backward pass.

The paper notes that newer cuDNN versions incorporated many of the techniques described in FlashAttention-4, which is why they reach similar performance in some settings (at least in the forward pass). But even then, the paper is really nice because it describes these optimizations in detail for those who are interested.

- 27 Jan, Quantization-Aware Distillation for NVFP4 Inference Accuracy Recovery, <https://arxiv.org/abs/2601.20088>
- 10 Feb, Efficient Remote KV Cache Reuse with GPU-Native Video Codec, <https://arxiv.org/abs/2602.09725>
- 13 Feb, SpargeAttention2: Trainable Sparse Attention via Hybrid Top-K+Top-P Masking and Distillation Fine-Tuning, <https://arxiv.org/abs/2602.13515>
- 20 Feb, Decoding as Optimisation on the Probability Simplex: From Top-K to Top-P (Nucleus) to Best-of-K Samplers, <https://arxiv.org/abs/2602.18292>
- 4 Mar, FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling, <https://arxiv.org/abs/2603.05451>

- 11 Mar, LookaheadKV: Fast and Accurate KV Cache Eviction by Glimpsing into the Future without Generation, <https://arxiv.org/abs/2603.10899>
- 17 Apr, Cut Your Losses! Learning to Prune Paths Early for Efficient Parallel Reasoning, <https://arxiv.org/abs/2604.16029>
- 12 May, A Queueing-Theoretic Framework for Stability Analysis of LLM Inference with KV Cache Memory Constraints, <https://arxiv.org/abs/2605.04595>

## 4. Sparse Attention and Long Context

The main theme this year is that long-context modeling is a main focus area, both for modeling and qualitative performance.

This section collects papers on sparse attention, selective state space attention, KV compression, long-context pre-training, and structured reasoning over long document sets.

Even though [\*DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence\*](#) could also fit into the architecture or inference-efficiency sections, I am including it here because its most interesting part for this overview is the new attention variant, which carries over some of the sparsity aspects from the DeepSeek V3.2 (DeepSeek Sparse Attention) mechanism.

(Of course, the report covers a lot more than attention, including a 1.6T-parameter MoE model, mHC residual connections, Muon optimization, post-training, infrastructure, and benchmark results.)

The main idea in DeepSeek V4 is to make sparse attention cheaper before the sparse lookup part. Earlier DeepSeek models used MLA-style compressed latent cache entries but still kept one entry per token. DeepSeek V4 instead compresses the history along the sequence dimension, so multiple original tokens become one compressed KV entry. This gives the sparse selector a much shorter history to search through.

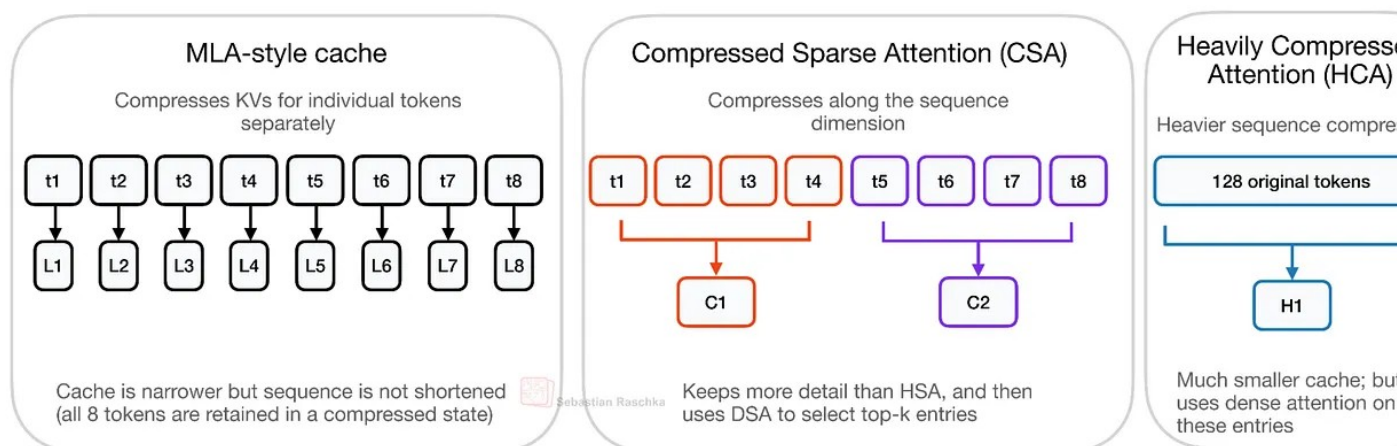
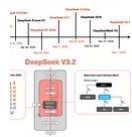


Figure 5: MLA compresses the KV representation for each token, but keeps one entry per token. CSA and HCA compress along the sequence dimension, so the sparse-attention problem gets smaller. For a longer explanation, see my previous article.

CSA is the key sparse-attention variant of the two, which first compresses every small block of tokens into a compressed KV entry. Then it uses a DeepSeek Sparse Attention-style selector to choose a top-k subset of these compressed history entries for the main attention computation. It also keeps a local sliding-window branch with recent raw KV entries, which helps preserve local detail.

The selector that makes the attention mask sparse is the Lightning Indexer. In simple terms, it is a lightweight attention-like scoring module that estimates which compressed history blocks are most relevant for the current query. The sparse part comes from the top-k step where after scoring the compressed KV entries, CSA keeps only a subset of them for the expensive core attention computation instead of attending over the whole compressed history. This is the same broad idea I discussed in my DeepSeek V3.2 article, where I explained the earlier DeepSeek sparse-attention setup in more detail:



## From DeepSeek V3 to V3.2: Architecture, Sparse Attention, and RL Updates

SEBASTIAN RASCHKA, PHD • DECEMBER 3, 2025

[Read full story →](#)

HCA is not a sparse attention variant but included in the figure below for completeness. It compresses a larger span of tokens, compared to CSA, into one cache entry and then uses dense attention over this much shorter compressed history. DeepSeek V4 then interleaves HCA with CSA, where the idea is that the sparse CSA part gives more selective access to compressed history, while the heavily compressed HCA part gives cheaper broad context access.

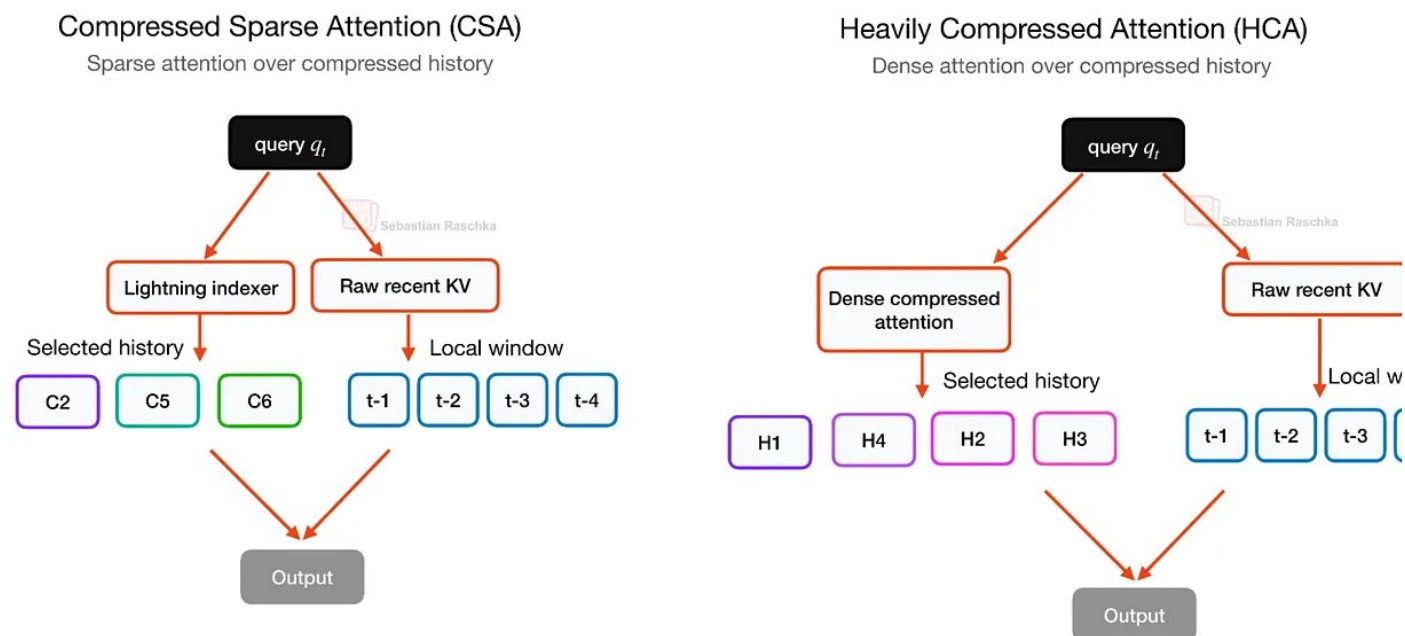


Figure 6: CSA selects a sparse set of compressed history blocks. HCA attends densely over more heavily compressed blocks. Both variants keep a local branch for recent uncompressed KV entries, which helps maintain short-range detail.

The reported efficiency numbers are large at one-million-token context length. DeepSeek V4-Pro uses 27 percent of the single-token inference FLOPs and 10 percent of the KV cache compared with DeepSeek V3.2. DeepSeek V4-Flash is even smaller, 10 percent of the FLOPs and 7 percent of the KV cache relative to DeepSeek V3.2.





Values are relative to DeepSeek V3.2 in the DeepSeek V4 paper, not absolute FLOP or cache values.

Figure 7: Reported one-million-token context efficiency numbers from the DeepSeek V4 report, relative to DeepSeek V3.2.

CSA is an interesting example of where sparse attention seems to be going. Instead selecting from the full raw history, the model first compresses the history, then sparsely selects from that compressed memory, and finally adds a local raw-token branch for recent context.

- 3 Feb, Token Sparse Attention: Efficient Long-Context Inference with Interleaved Token Selection, <https://arxiv.org/abs/2602.03216>
- 12 Mar, IndexCache: Accelerating Sparse Attention via Cross-Layer Index Reuse <https://arxiv.org/abs/2603.12201>
- 16 Mar, Mixture-of-Depths Attention, <https://arxiv.org/abs/2603.15619>
- 6 Apr, TriAttention: Efficient Long Reasoning with Trigonometric KV Compression <https://arxiv.org/abs/2604.04921>
- 20 Apr, Sessa: Selective State Space Attention, <https://arxiv.org/abs/2604.18580>
- 24 Apr, Contexts Are Never Long Enough: Structured Reasoning for Scalable Question Answering over Long Document Sets, <https://arxiv.org/abs/2604.22290>
- 5 May, DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence [https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek\\_V4.pdf](https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf)
- 7 May, Long Context Pre-Training with Lighthouse Attention, <https://arxiv.org/abs/2605.06554>

- 16 May, Full Attention Strikes Back: Transferring Full Attention into Sparse with Hundred Training Steps, <https://arxiv.org/abs/2605.16928>

## 5. Reasoning and Test-Time Compute

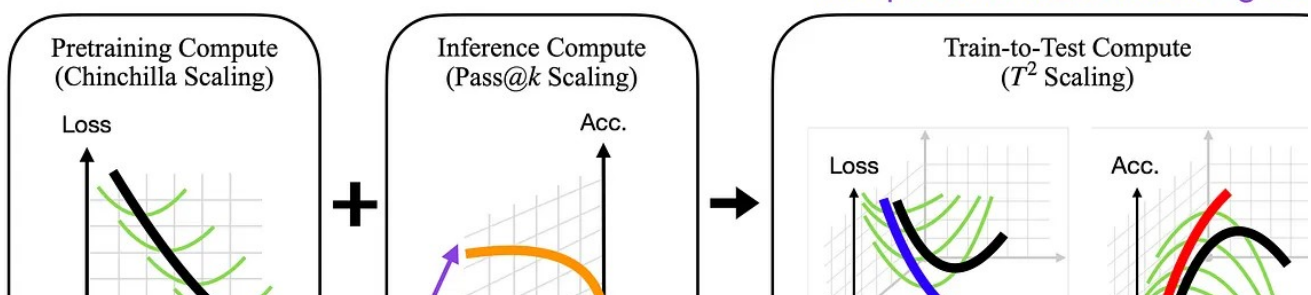
Following OpenAI's o1 and DeepSeek R1, reasoning models were one of the biggest themes of 2025, and they are still a central theme in 2026.

There are still many papers that propose variants and improvements to the RLVR-related training techniques (covered in the next section), but below, I focused on papers that look at when to spend test-time compute, when to stop thinking, and how to combine reasoning with agentic workflows.

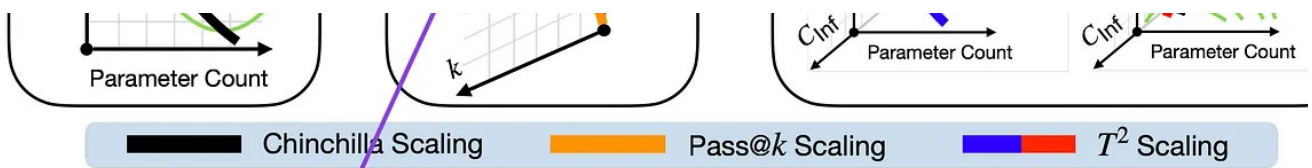
**By the way, I have been hard at work finishing the final revisions and layouts of my new [Build A Reasoning \(From Scratch\)](#) book, which covers both training and inference-/test-time scaling techniques. It was just sent to print this week and will be available soon (the paperback will be in color this time).**

One particularly interesting pick here is the [Test-Time Scaling Makes Overtraining Compute-Optimal](#) paper. It is a nice conceptual paper because it connects pre-train compute with inference-time compute. Standard scaling laws usually ask how large model should be and how many tokens it should see for a fixed training budget. And this paper builds on these but adds another variable on top: how many samples or attempts we plan to spend at test time.

### Classic scaling laws



Proposed scaling laws recommend overtraining compared to classic scaling law



Repeated sampling  $k$  times  
during inference (test-time  
scaling) improves accuracy

Figure 8: Overview figure from the *Test-Time Scaling Makes Overtraining Compute-Optimal* paper. Train-to-Test scaling combines Chinchilla-style pre-training scaling with pass@ $k$ -style test-time scaling.

The main takeaway is that test-time scaling can make smaller, more heavily trained models compute-optimal. Under standard scaling-laws, this looks like overtraining because the model sees more tokens than the classic scaling laws would suggest. But we use test-time compute scaling like self-consistency sampling (aka pass@ $k$ ), a smaller model can be cheaper to sample many times than to sample a larger model a smaller number of times.

The caveat is that the result depends on the task and inference strategy. But for settings that rely on pass@ $k$  and other inference-/test-time scaling techniques, it is useful insight that they formalized here, namely that pre-training and inference should be optimized together.

- 9 Jan, PaCoRe: Learning to Scale Test-Time Compute with Parallel Coordinated Reasoning, <https://arxiv.org/abs/2601.05593>
- 4 Feb, Learning to Reason in 13 Parameters, <https://arxiv.org/abs/2602.04118>
- 6 Feb, InftyThink+: Effective and Efficient Infinite-Horizon Reasoning via Reinforcement Learning, <https://arxiv.org/abs/2602.06960>
- 7 Feb, Learning to Self-Verify Makes Language Models Better Reasoners, <https://arxiv.org/abs/2602.07594>
- 9 Feb, Does Your Reasoning Model Implicitly Know When to Stop Thinking?,

<https://arxiv.org/abs/2602.08354>

- 5 Mar, Reasoning Models Struggle to Control Their Chains of Thought, <https://arxiv.org/abs/2603.05706>
- 10 Mar, Thinking to Recall: How Reasoning Unlocks Parametric Knowledge in LLMs, <https://arxiv.org/abs/2603.09906>
- 18 Mar, Nemotron-Cascade 2: Post-Training LLMs with Cascade RL and Multi-Domain On-Policy Distillation, <https://arxiv.org/abs/2603.19220>
- 1 Apr, Test-Time Scaling Makes Overtraining Compute-Optimal, <https://arxiv.org/abs/2604.01411>
- 1 Apr, Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets, <https://arxiv.org/abs/2604.024>
- 20 Apr, When Can LLMs Learn to Reason with Weak Supervision?, <https://arxiv.org/abs/2604.18574>
- 6 May, AI Co-Mathematician: Accelerating Mathematicians with Agentic AI, <https://arxiv.org/abs/2605.06651>
- 8 May, LLMs Improving LLMs: Agentic Discovery for Test-Time Scaling, <https://arxiv.org/abs/2605.08083>
- 13 May, Achieving Gold-Medal-Level Olympiad Reasoning via Simple and Unified Scaling, <https://arxiv.org/abs/2605.13301>
- 26 May, Share More, Search Less: Collaborative Parallel Thinking for Efficient Test-Time Scaling, <https://arxiv.org/abs/2605.27030>

## 6. Reinforcement Learning and RLVR

This section includes papers on training reasoning models, that is, reinforcement learning with verifiable reward (RLVR) for reasoning, self-distillation, policy optimization, text feedback, and multimodal chain-of-thought training.

The broader trend is that RLVR is now a familiar technique, so, many papers are therefore about making RL more stable, cheaper, more general, or easier to apply beyond narrowly verifiable tasks.

One interesting pick here is [You Only Need Minimal RLVR Training: Extrapolating LLM via Rank-1 Trajectories](#), which looks into what RLVR training actually does to the model weights, instead of only proposing another policy optimization variant.

The main finding is that RLVR weight updates appear to be much more structured than one might expect. The authors save checkpoints during GRPO training and look at the parameter deltas relative to the base model. Then, for each weight tensor, they stack the deltas over training time and run a singular value decomposition (SVD) to identify the dominant directions along which the weights move during RLVR training.

For those unfamiliar with SVD, in principle, SVD is related to Principal Component Analysis, which is about finding the main directions of variation in high-dimensional data. Here, the “data” are the weight changes over training, so SVD helps identify the dominant directions in which the model parameters move during RLVR.

Surprisingly, a rank-1 direction per tensor preserves most of the downstream-relevant change. In other words, much of the useful RLVR update seems to move along one dominant direction.

They then add a second observation. Once the updates are projected onto this dominant rank-1 direction, the scalar coefficient changes almost linearly with training step. This leads to their RELEX method, that is, train RLVR for a short prefix, estimate the rank-1 direction, fit a line to the coefficient trajectory, and extrapolate a future checkpoint without running the remaining RLVR steps. (Concretely, for each weight tensor, they estimate a dominant direction ( $v_1$ ) from the early RLVR checkpoints. Then they project each observed weight update onto that direction.)

Raw Weight Space  
Raw weight trajectory during RLVR

SVD Weight Space  
RELEX leverages early training dynamics

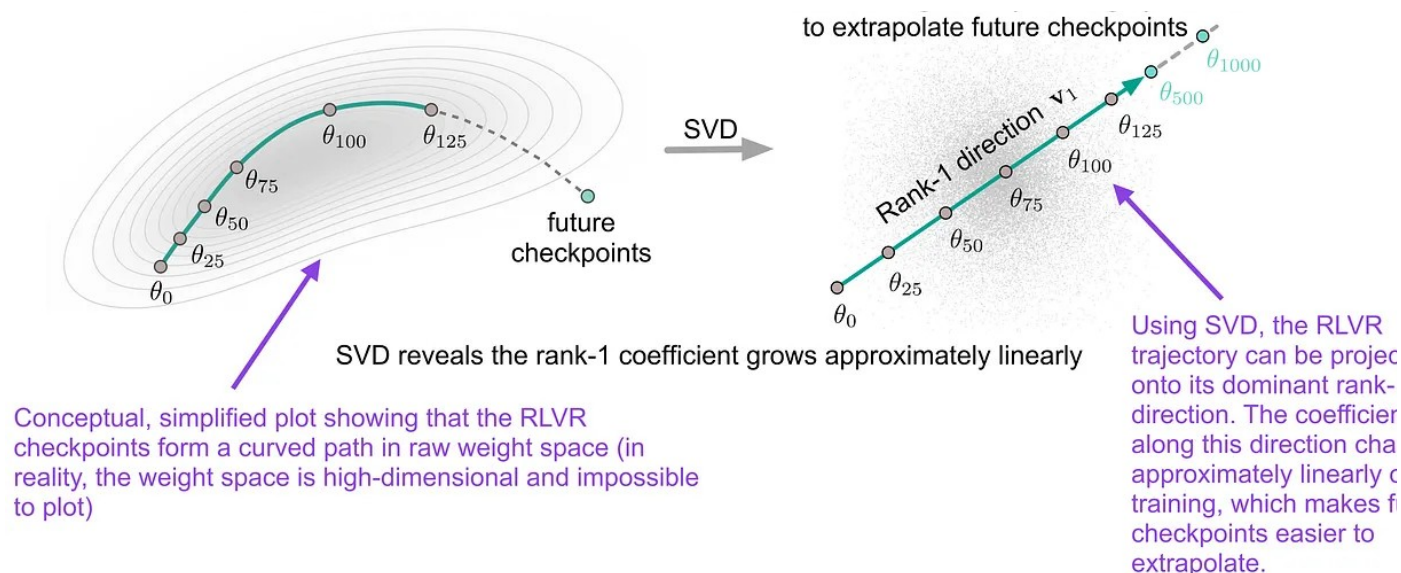


Figure 9: Method overview from the *You Only Need Minimal RLVR Training* paper. Raw RLVR checkpoints trace a curved path in weight space. After SVD, the dominant rank-1 direction captures most of the useful movement, and the coefficient along that direction can be extrapolated.

Impressively, in their experiments, RELEX uses only 15-20% of the full RLVR training steps and matches or nearly matches full RLVR on Qwen2.5-Math-1.5B, Qwen3-4B-Base, and Qwen3-8B-Base on MATH.

While the experiments look extremely promising, I remain a bit skeptical here that it will become a general, widely used approach. However, I think it's a super interesting diagnostic paper that gives us some insights into one aspect of the RLVR training behavior.

- 8 Jan, GDPO: Group Reward-Decoupled Normalization Policy Optimization for Multi-Reward RL Optimization, <https://arxiv.org/abs/2601.05242>
- 13 Jan, Your Group-Relative Advantage Is Biased, <https://arxiv.org/abs/2601.08108>
- 15 Jan, Reasoning Models Generate Societies of Thought, <https://arxiv.org/abs/2601.10825>
- 26 Jan, Teaching Models to Teach Themselves: Reasoning at the Edge of Learnability, <https://arxiv.org/abs/2601.18778>
- 28 Jan, Harder Is Better: Boosting Mathematical Reasoning via Difficulty-Aware

GRPO and Multi-Aspect Question Reformulation, <https://arxiv.org/abs/2601.20802>

- 28 Jan, Reinforcement Learning via Self-Distillation, <https://arxiv.org/abs/2601.20802>
- 2 Feb, Expanding the Capabilities of Reinforcement Learning via Text Feedback, <https://arxiv.org/abs/2602.02482>
- 9 Feb, iGRPO: Self-Feedback-Driven LLM Reasoning, <https://arxiv.org/abs/2602.09000>
- 11 Feb, Data Repetition Beats Data Scaling in Long-CoT Supervised Fine-Tuning, <https://arxiv.org/abs/2602.11149>
- 14 Feb, Experiential Reinforcement Learning, <https://arxiv.org/abs/2602.13949>
- 17 Feb, Discovering Implicit Large Language Model Alignment Objectives, <https://arxiv.org/abs/2602.15338>
- 10 Mar, OpenClaw-RL: Train Any Agent Simply by Talking, <https://arxiv.org/abs/2603.10165>
- 23 Mar, Rethinking Token-Level Policy Optimization for Multimodal Chain-of-Thought, <https://arxiv.org/abs/2603.22847>
- 3 Apr, Self-Distilled RLVR, <https://arxiv.org/abs/2604.03128>
- 7 Apr, Target Policy Optimization, <https://arxiv.org/abs/2604.06159>
- 8 Apr, FP4 Explore, BF16 Train: Diffusion Reinforcement Learning via Efficient Rollout Scaling, <https://arxiv.org/abs/2604.06916>
- 9 Apr, SPPO: Sequence-Level PPO for Long-Horizon Reasoning Tasks, <https://arxiv.org/abs/2604.08865>
- 15 Apr, Reward Hacking in the Era of Large Models: Mechanisms, Emergent Misalignment, Challenges, <https://arxiv.org/abs/2604.13602>
- 8 May, SEIF: Self-Evolving Reinforcement Learning for Instruction Following, <https://arxiv.org/abs/2605.07465>

- 14 May, Process Rewards with Learned Reliability, <https://arxiv.org/abs/2605.15155>
- 14 May, Self-Distilled Agentic Reinforcement Learning, <https://arxiv.org/abs/2605.15155>
- 20 May, You Only Need Minimal RLVR Training: Extrapolating LLMs via Rank-1 Trajectories, <https://arxiv.org/abs/2605.21468>
- 25 May, DVAO: Dynamic Variance-adaptive Advantage Optimization for Multi-reward Reinforcement Learning, <https://arxiv.org/abs/2605.25604>
- 27 May, Agent Explorative Policy Optimization for Multimodal Agentic Reasoning, <https://arxiv.org/abs/2605.28774>
- 30 May, Trust Region On-Policy Distillation, <https://arxiv.org/abs/2606.01249>

## 7. Agent Systems and Tool Use

Agent papers are a large part of my list this year, which is probably not too surprising. (By the way, I do plan to follow up with a dedicated agent article, since there are a lot of interesting subcategories to discuss.)

One interesting paper here is [\*Is Grep All You Need? How Agent Harnesses Reshape Agentic Search\*](#). I like this paper because it studies a very practical question that comes up all the time when building agents / agent harnesses. Should an agent use fancy vector search (think RAG) in semantic space, or can simple lexical search such as `grep` be enough?

The paper compares `grep`-style lexical search and vector search on a 116-question subset of [LongMemEval](#). This benchmark then asks questions over long multi-session conversations, so the agent has to retrieve the right pieces of prior context before answering. The authors test this across a custom Chronos harness and provider-native CLI-style harnesses such as Claude Code, Codex CLI, and Gemini CLI.

The paper also varies how tool results are delivered to the model. In one setting,

results are inserted inline into the model context, etc.

It turns out that good old grep works surprisingly well!

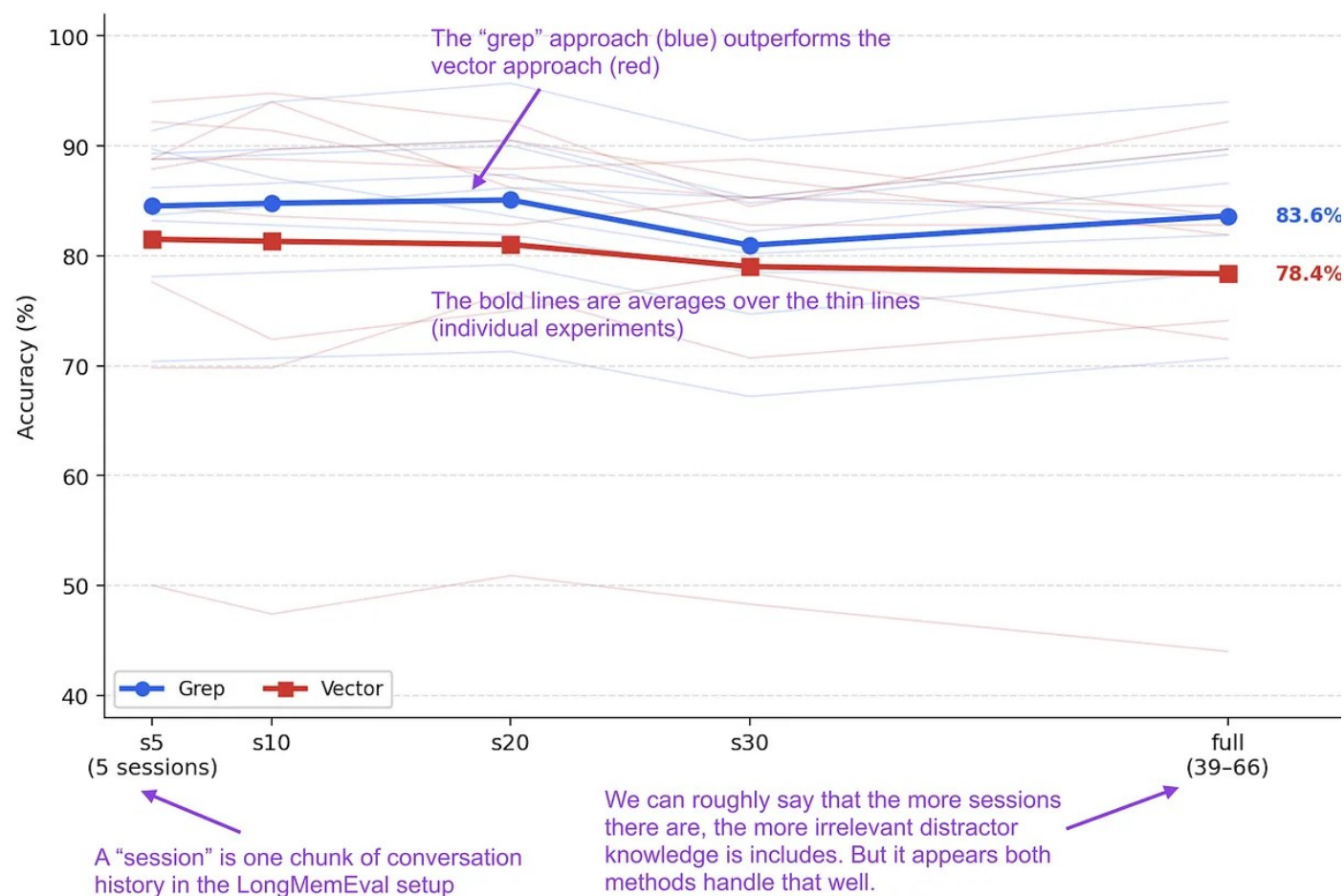


Figure 10: Figure from the *Is Grep All You Need?* paper (<https://arxiv.org/abs/2605.15184>). The plot compares grep and vector search as more irrelevant sessions are added to the retrieval pool. The thicker lines show average performance across settings, while the lighter lines show individual runs.

The results shown in Figure 10 are pretty clear, but it doesn't mention efficiency. Sure, grep itself is cheap because it does not require embeddings, an index, or approximate nearest-neighbor search, etc. But if the matching results are long and inserted directly into the model context, the overall agent run can still become expensive because the context gets longer.

The paper kind of studies this by comparing inline result delivery with a programmatic file-based setup, where search results are written to disk and the agent receives only

file path or pointer. This reduces context pressure, but it also adds extra tool-use steps because the model has to explicitly inspect the file. Again, the question is whether this has impacts on efficiency.

Either way, I find the results quite convincing and relevant, and it's nice to see that simpler methods perform quite well. This doesn't mean that vector search is dead (this paper only looks at a small slice of tasks or benchmarks), but my takeaway here is that vector search can sometimes or often be overkill (I have similar views about MCP).

- 11 Jan, Dr. Zero: Self-Evolving Search Agents Without Training Data, <https://arxiv.org/abs/2601.07055>
- 18 Jan, Rethinking the Value of Multi-Agent Workflow: A Strong Single Agent Baseline, <https://arxiv.org/abs/2601.12307>
- 30 Jan, Large Language Model Agents Are Not Always Faithful Self-Evolvers, <https://arxiv.org/abs/2601.22436>
- 3 Feb, Agent Primitives: Reusable Latent Building Blocks for Multi-Agent Systems, <https://arxiv.org/abs/2602.03695>
- 3 Feb, AgentArk: Distilling Multi-Agent Intelligence into a Single LLM Agent, <https://arxiv.org/abs/2602.03955>
- 12 Feb, Gaia2: Benchmarking LLM Agents on Dynamic and Asynchronous Environments, <https://arxiv.org/abs/2602.11964>
- 13 Feb, SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks, <https://arxiv.org/abs/2602.12670>
- 15 Feb, Does Socialization Emerge in AI Agent Society? A Case Study of Moltb, <https://arxiv.org/abs/2602.14299>
- 18 Feb, AgentLAB: Benchmarking LLM Agents Against Long-Horizon Attacks, <https://arxiv.org/abs/2602.16901>
- 22 Feb, Learning to Rewrite Tool Descriptions for Reliable LLM-Agent Tool Use, <https://arxiv.org/abs/2602.20426>

- 24 Feb, Evaluating Theory of Mind and Internal Beliefs in LLM-Based Multi-Age Systems, <https://arxiv.org/abs/2603.00142>
- 4 Mar, AgentIR: Reasoning-Aware Retrieval for Deep Research Agents, <https://arxiv.org/abs/2603.04384>
- 18 Mar, Hyperagents, <https://arxiv.org/abs/2603.19461>
- 19 Mar, Multi-User Large Language Model Agents, <https://arxiv.org/abs/2604.08567>
- 23 Mar, From Static Templates to Dynamic Runtime Graphs: A Survey of Workflow Optimization for LLM Agents, <https://arxiv.org/abs/2603.22386>
- 25 Mar, Natural-Language Agent Harnesses, <https://arxiv.org/abs/2603.25723>
- 30 Mar, Drop the Hierarchy and Roles: How Self-Organizing LLM Agents Outperform Designed Structures, <https://arxiv.org/abs/2603.28990>
- 4 Apr, The Art of Building Verifiers for Computer Use Agents, <https://arxiv.org/abs/2604.06240>
- 9 Apr, Externalization in LLM Agents: A Unified Review of Memory, Skills, Protocol and Harness Engineering, <https://arxiv.org/abs/2604.08224>
- 15 Apr, Autogenesis: A Self-Evolving Agent Protocol, <https://arxiv.org/abs/2604.15034>
- 23 Apr, From Skills to Talent: Organising Heterogeneous Agents as a Real-World Company, <https://arxiv.org/abs/2604.22446>
- 24 Apr, Agentic World Modeling: Foundations, Capabilities, Laws, and Beyond, <https://arxiv.org/abs/2604.22748>
- 26 Apr, From Skill Text to Skill Structure: The Scheduling-Structural-Logical Representation for Agent Skills, <https://arxiv.org/abs/2604.24026>
- 26 Apr, Latent Agents: A Post-Training Procedure for Internalized Multi-Agent Debate, <https://arxiv.org/abs/2604.24881>
- 28 Apr, Recursive Multi-Agent Systems, <https://arxiv.org/abs/2604.25917>

- 29 Apr, ClawGym: A Scalable Framework for Building Effective Claw Agents, <https://arxiv.org/abs/2604.26904>
- 30 Apr, Skills as Verifiable Artifacts: A Trust Schema and a Biconditional Correctness Criterion for Human-in-the-Loop Agent Runtimes, <https://arxiv.org/abs/2605.00424>
- 13 May, Is Grep All You Need? How Agent Harnesses Reshape Agentic Search, <https://arxiv.org/abs/2605.15184>
- 17 May, AI for Auto-Research: Roadmap & User Guide, <https://arxiv.org/abs/2605.18661>
- 19 May, A Methodology for Selecting and Composing Runtime Architecture Patterns for Production LLM Agents, <https://arxiv.org/abs/2605.20173>
- 20 May, Compiling Agentic Workflows into LLM Weights: Near-Frontier Quality Two Orders of Magnitude Less Cost, <https://arxiv.org/abs/2605.22502>
- 22 May, SkillOpt: Executive Strategy for Self-Evolving Agent Skills, <https://arxiv.org/abs/2605.23904>
- 24 May, Your Agents Are Aging Too: Agent Lifespan Engineering for Deployed Systems, <https://arxiv.org/abs/2605.26302>

## 8. Coding Agents and Software Engineering

I separated coding agents from the broader agent category because this has become a substantial topic on its own.

This section includes papers on coding benchmarks, terminal capabilities, repository level context files, harness optimization, and long-context coding workflows.

One paper that definitely got my attention is [Evaluating AGENTS.md: Are Repository Level Context Files Helpful for Coding Agents?](#). It looks into whether adding repository level instruction files such as `AGENTS.md` or `CLAUDE.md` to tell coding agents how to

work in a codebase is actually helpful.

The paper evaluates this in two settings. First, it uses SWE-bench Lite, where the authors generate context files because the original repositories do not necessarily contain developer-written ones. Second, they introduce AGENTBENCH, a new benchmark of 138 Python tasks from 12 repositories that already have developer-provided context files. The agents are then evaluated under three conditions: no context file, an LLM-generated context file, and, where available, a developer-written context file. The results are summarized below.

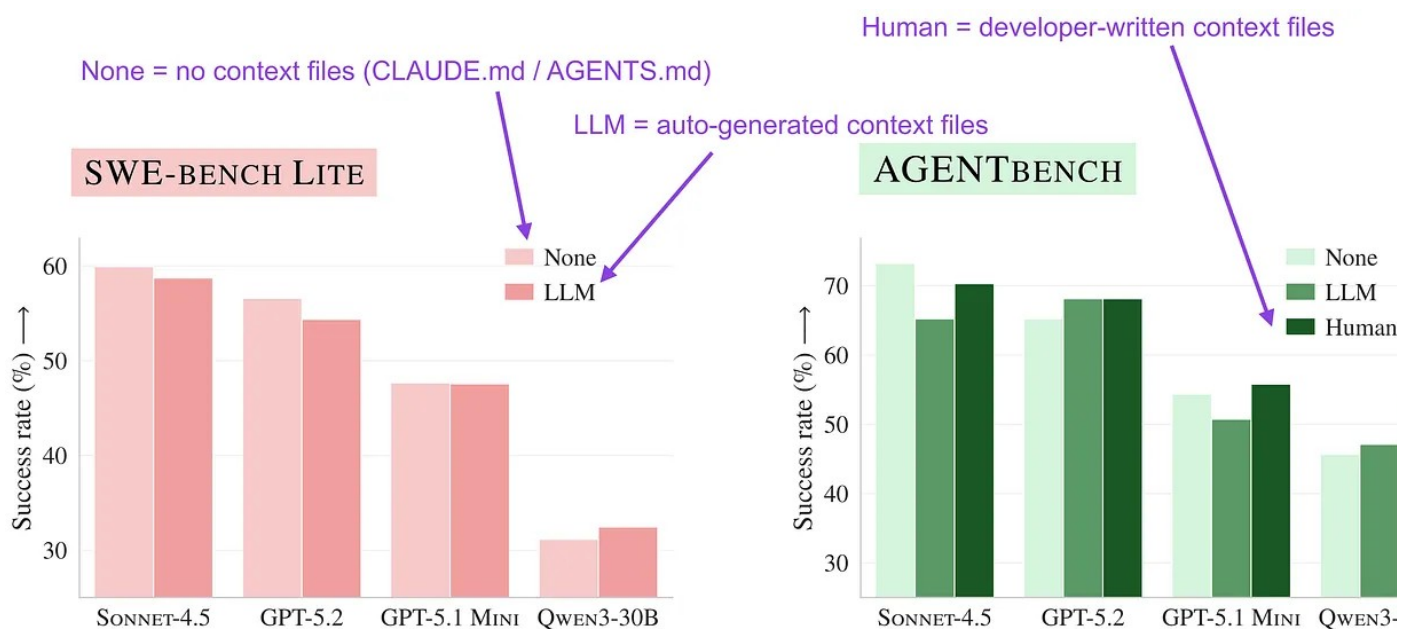


Figure 11: Main results from the *Evaluating AGENTS.md* paper (<https://arxiv.org/abs/2602.11988>).

Based on the results shown in the figure above, compared to using no context files, LLM-generated context files reduce task success slightly or don't make a big difference on average. This is maybe surprising but maybe not, because I guess the LLM / agent harness just generates the necessary context information on the fly. The context file is more about improving efficiency between independent sessions.

Also, developer-written context files are better than LLM-generated ones, which is perhaps expected because that's where the domain expertise comes in.

What's very surprising though is that using no context files is also cheaper and more efficient in their benchmarks!

**Table 2.** The average number of steps (lower is better) and execution cost (in USD — lower is better) per SWE-BENCH LITE and AGENTBENCH instance without context files (NONE), with LLM-generated context files (LLM), and with developer-written context files (HUM). We **bold** the best setting.

|                | Type | SONNET-4.5  |             | GPT-5.2     |             | GPT-5.1 M.  |             | QWEN3-30B   |             |
|----------------|------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
|                |      | Steps       | Cost        | Steps       | Cost        | Steps       | Cost        | Steps       | Cost        |
| SWE-BENCH LITE | NONE | <b>54.4</b> | <b>1.30</b> | <b>12.5</b> | <b>0.32</b> | <b>40.9</b> | <b>0.18</b> | <b>29.7</b> | <b>0.12</b> |
|                | LLM  | 57.2        | 1.51        | 12.7        | 0.43        | 45.2        | 0.22        | 32.2        | 0.13        |
| AGENT-BENCH    | NONE | <b>40.7</b> | <b>1.15</b> | <b>12.1</b> | <b>0.38</b> | <b>40.6</b> | <b>0.18</b> | <b>31.5</b> | <b>0.13</b> |
|                | LLM  | 46.5        | 1.33        | 13.1        | 0.57        | 46.9        | 0.20        | 34.2        | 0.15        |
|                | HUM. | 45.3        | 1.30        | 13.6        | 0.54        | 46.6        | 0.19        | 32.8        | 0.15        |

Figure 12: Efficiency results from the *Evaluating AGENTS.md* paper (<https://arxiv.org/abs/2602.11988>).

The fact that using no context files results in better efficiency is a bit mind-boggling first. I first suspected this is perhaps because the harnesses here might be processing redundant information (i.e., they read the context files, and, no matter what, they pass the additional info from the code repo as if they hadn't read the context files).

The researchers did a trace analysis here that showed that the agents generally follow the instructions in the context files, but they run more tests, search more files, read more files, and use more repository-specific tools when those tools are mentioned. The negative or weak performance effect does not seem to come from agents ignoring the files. A more likely explanation is that context files often add requirements and exploration steps that make the task harder or more thorough, but as we saw in Fig 11, this doesn't necessarily result in better success.

My takeaway is that repository-level context files should probably be kept shorter and more specific and perhaps ideally hierarchical (e.g., "if you do x, check this other

context file y.md, otherwise ignore it").

Of course, the problem here is that the LLMs and harnesses are a bit dated by now, and it would be interesting to redo this study with the latest harnesses and LLMs.

- 19 Jan, AI IDEs or Autonomous Agents? Measuring the Impact of Coding Agent on Software Development, <https://arxiv.org/abs/2601.13597>
- 2 Feb, CodeOCR: On the Effectiveness of Vision Language Models in Code Understanding, <https://arxiv.org/abs/2602.01785>
- 10 Feb, AutoHarness: Improving LLM Agents by Automatically Synthesizing a C Harness, <https://arxiv.org/abs/2603.03329>
- 11 Feb, Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?, <https://arxiv.org/abs/2602.11988>
- 15 Feb, LongCLI-Bench: A Preliminary Benchmark and Study for Long-Horizon Agentic Programming in Command-Line Interfaces, <https://arxiv.org/abs/2602.14337>
- 24 Feb, On Data Engineering for Scaling LLM Terminal Capabilities, <https://arxiv.org/abs/2602.21193>
- 27 Feb, SWE-rebench V2: Language-Agnostic SWE Task Collection at Scale, <https://arxiv.org/abs/2602.23866>
- 28 Feb, Qwen3-Coder-Next Technical Report, <https://arxiv.org/abs/2603.00729>
- 3 Mar, BeyondSWE: Can Current Code Agent Survive Beyond Single-Repo Bug Fixing?, <https://arxiv.org/abs/2603.03194>
- 19 Mar, Coding Agents Are Effective Long-Context Processors, <https://arxiv.org/abs/2603.20432>
- 22 Mar, Effective Strategies for Asynchronous Software Engineering Agents, <https://arxiv.org/abs/2603.21489>
- 29 Mar, Meta-Harness: End-to-End Optimization of Model Harnesses, <https://arxiv.org/abs/2603.21489>

[arxiv.org/abs/2603.28052](https://arxiv.org/abs/2603.28052)

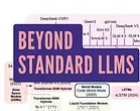
- 6 Apr, Scaling Coding Agents via Atomic Skills, <https://arxiv.org/abs/2604.05013>
- 26 Apr, Frontier Coding Agents Can Now Implement an AlphaZero Self-Play Machine Learning Pipeline for Connect Four That Performs Comparably to an External Solver, <https://arxiv.org/abs/2604.25067>
- 27 Apr, Agentic Harness Engineering: Observability-Driven Automatic Evolution Coding-Agent Harnesses, <https://arxiv.org/abs/2604.25850>
- 18 May, Code as Agent Harness, <https://arxiv.org/abs/2605.18747>

## 9. Diffusion Language Models

Diffusion language models were already a recurring topic in 2025, and they continue to appear in 2026. (I wouldn't say that they are popular, yet, though. Also, I wonder whatever happened to [Gemini Diffusion](#). I have been waiting.)

Anyways, I am still watching this subfield with some interest because it explores a different generation paradigm from standard left-to-right autoregressive decoding.

You can find a bit more background on text diffusion models in my Beyond Standard LLMs article:



### Beyond Standard LLMs

SEBASTIAN RASCHKA, PHD • NOVEMBER 4, 2025

[Read full story →](#)

In short, the basic idea behind diffusion language modeling can be summarized as follows. A (non-diffusion) autoregressive language model generates text from left to right. A diffusion language model starts from a corrupted or masked sequence and iteratively denoises it. In masked diffusion, tokens are randomly replaced by mask tokens during training, and the model learns to recover the original clean tokens. At inference time, the model can fill in multiple positions over several denoising steps,

instead of committing to one next token at a time.

One major downside still is that it's tricky to use them in modern agent and coding harnesses due to their parallel generation nature. For instance, when we carry out a depend on tool calls, the generation is better suited as a sequential setup.

Also, the tool-call shortcoming aside, the main questions still are

- Can these models become competitive on quality?
- Can they provide meaningful speedups at similar quality?
- And can they be integrated into existing LLM workflows without adding too much complexity?

To continue with the theme of selecting one paper per section, one interesting paper here is [LLaDA2.1: Speeding Up Text Diffusion via Token Editing](#). It investigates the speed aspect and asks whether a diffusion language model can use parallel decoding more aggressively without locking in too many bad early choices.

In regular masked diffusion, the model replaces mask tokens with predicted tokens. Once a mask is filled, that token is usually treated as fixed. This can be brittle because if the model fills many positions in parallel, one bad early token can become part of the context for the next denoising step.

To address this fixed-token issue, LLaDA2.1 adds a second update type. Besides Mask-to-Token (M2T) updates, where a masked position becomes a token, it also uses Token-to-Token (T2T) updates, where an already-filled token can be replaced by a better token later. In other words, the model can draft quickly and then revise.

The decoding rule is controlled by two confidence thresholds. One threshold decides when a mask should be filled. The other decides when an existing token should be edited. Lowering the mask-filling threshold makes decoding faster because more tokens are generated per step. The editing threshold then gives the model a way to

repair some of the mistakes introduced by this more aggressive drafting. This leads (1) a Speedy Mode that drafts more aggressively and relies more on edits, and (2) a Quality Mode that uses more conservative thresholds.

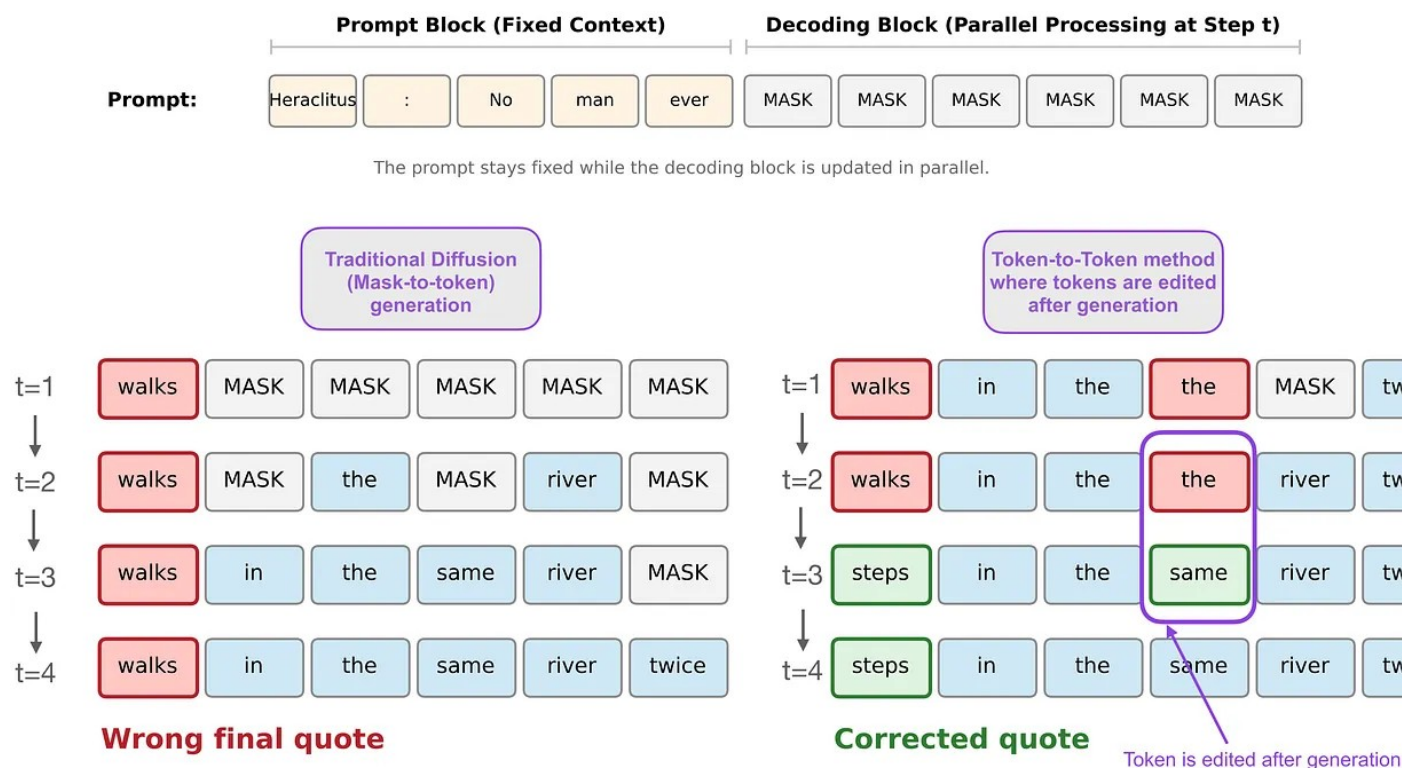


Figure 13: Illustration of the token-editing concept in the LLaDA2.1 paper (<https://arxiv.org/abs/2602.08676>).

While this is definitely an important improvement over standard diffusion modeling note that this does not remove the integration issue for agentic workflows where generation often has to proceed sequentially around tool calls. But for ordinary text code, and math generation, which doesn't require tool calls, this is a nice example c how diffusion LMs can turn their parallel generation setup into a more practical spe quality tradeoff.

- 5 Jan, Deferred Commitment Decoding for Diffusion Language Models, <https://arxiv.org/abs/2601.02076>
- 4 Feb, Generative Modeling via Drifting, <https://arxiv.org/abs/2602.04770>
- 9 Feb, LLaDA2.1: Speeding Up Text Diffusion via Token Editing, <https://arxiv.org>,

[abs/2602.08676](https://arxiv.org/abs/2602.08676)

- 26 Feb, dLLM: Simple Diffusion Language Modeling, <https://arxiv.org/abs/2602.22661>
- 4 Apr, Rethinking Token Prediction: Tree-Structured Diffusion Language Model, <https://arxiv.org/abs/2604.03537>
- 7 May, Continuous Latent Diffusion Language Model, <https://arxiv.org/abs/2605.06548>
- 8 May, How to Train Your Latent Diffusion Language Model Jointly with the Latent Space, <https://arxiv.org/abs/2605.07933>
- 8 May, TextLDM: Language Modeling with Continuous Latent Diffusion, <https://arxiv.org/abs/2605.07748>
- 14 May, Factorization-Error-Free Discrete Diffusion Language Model via Speculative Decoding, <https://arxiv.org/abs/2605.14305>

## 10. Model Evaluation and Benchmarks

It is easy to get impressed by model releases and fancy new techniques, but the usefulness of these depends heavily on whether the evaluations are comprehensive set up correctly, and ultimately measure the right thing (for example, better scores on SWE-Bench don't necessarily translate to better model performance for the end use on real-world coding tasks.)

The papers below cover agent evaluation, benchmark validity, adversarial attacks, long-horizon tasks, structured extraction, and reasoning failures.

For instance, *AlphaEval: Evaluating Agents in Production* (<https://arxiv.org/abs/2604.12162>) looks at agent evaluation closer to how these systems are actually used. Usually, many agent benchmarks are built from clean, retrospective tasks with explicit requirements and deterministic success criteria. Having worked in both academic research and industry, I can confirm that production work is messier.

AlphaEval is a production-grounded benchmark with 94 tasks sourced from seven companies across six occupational domains. A key detail is that it evaluates complete agent harnesses, such as Claude Code, Codex, GitHub Copilot, and Cursor, rather than only the underlying model.

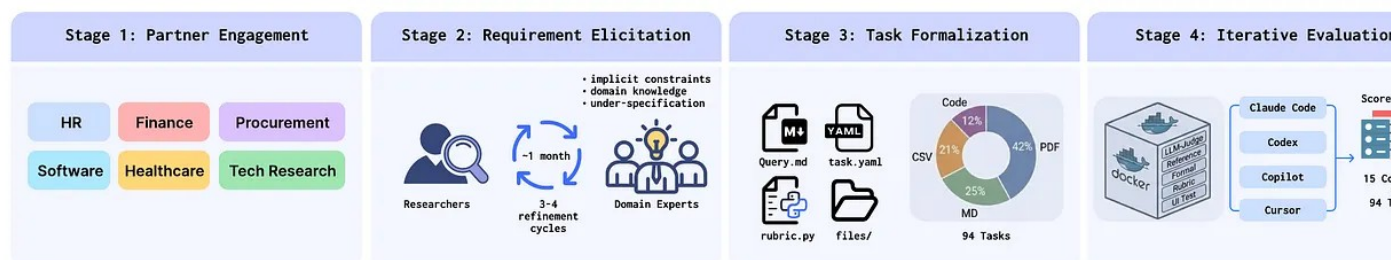


Figure 14: Requirement-to-benchmark construction pipeline from the AlphaEval paper (<https://arxiv.org/abs/2604.12162>).

The results also once more confirm my hunch that model performance (because proprietary model developers heavily fine-tune their models to perform well in their own harness) really depends on choosing the right harness. For instance, the best configuration in the paper, Claude Code with Opus 4.6, reaches 64.41 out of 100. The same Opus 4.6 model scores 53.45 when used through Codex in their setup, which is an 11-point difference.

I want to emphasize that this doesn't imply that Claude Code is a better harness than Codex, but that Opus has (more likely) been fine-tuned for Claude Code.

- 7 Jan, Benchmark<sup>2</sup>: Systematic Evaluation of LLM Benchmarks, <https://arxiv.org/abs/2601.03986>
- 5 Feb, OdysseyArena: Benchmarking Large Language Models for Long-Horizon Active and Inductive Interactions, <https://arxiv.org/abs/2602.05843>
- 5 Feb, Large Language Model Reasoning Failures, <https://www.arxiv.org/abs/2602.06176>
- 8 Feb, Emergent Misalignment Is Easy, Narrow Misalignment Is Hard, <https://arxiv.org/abs/2602.07852>

- 11 Feb, Maximal Brain Damage Without Data or Optimization: Disrupting Neural Networks via Sign-Bit Flips, <https://arxiv.org/abs/2502.07408>
- 16 Feb, LLMStructBench: Benchmarking Large Language Model Structured Data Extraction, <https://arxiv.org/abs/2602.14743>
- 17 Feb, Quantifying Construct Validity in Large Language Model Evaluations, <https://arxiv.org/abs/2602.15532>
- 5 Mar, Lost in Stories: Consistency Bugs in Long Story Generation by LLMs, <http://arxiv.org/abs/2603.05890>
- 23 Mar, ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence, <https://arxiv.org/abs/2603.24621>
- 24 Mar, Claudini: Autoresearch Discovers State-of-the-Art Adversarial Attack Algorithms for LLMs, <https://arxiv.org/abs/2603.24511>
- 5 Apr, ClawArena: Benchmarking AI Agents in Evolving Information Environments, <https://arxiv.org/abs/2604.04202>
- 13 Apr, AlphaEval: Evaluating Agents in Production, <https://arxiv.org/abs/2604.12162>
- 3 May, Reward Hacking Benchmark: Measuring Exploits in LLM Agents with Tool Use, <https://arxiv.org/abs/2605.02964>

## Conclusion

I hope you found this list useful as a personal reference, and it gives you a glimpse of the recent developments on the research front.

Looking across the papers, there is still plenty of work on larger and stronger models. Also, a lot of the interesting progress is now about using compute more selectively (sparse attention, KV-cache improvements, faster attention kernels, more targeted data pipelines, training methods that try to get more out of the same model).

Another recurring theme is that model quality increasingly depends on the surrounding system (agent harnesses, tool-use data, inference-time sampling). That makes the boundary between “model research” and “systems research” blurrier than used to be.

**Thanks for reading and for supporting Ahead of AI. These curated lists take time to maintain, but I find them useful myself, and I hope they help you find a handful of papers worth reading in more detail.**



### Recommend Ahead of AI to your readers

Ahead of AI focuses on machine learning and AI research and is read by more than 200,000 researchers and practitioners who want to stay ahead in a rapidly evolving field.

Recommend



86 Likes · 11 Restacks

### Discussion about this post

Comments   Restacks



Write a comment...



Dima 🌟 Dima Jun 7

❤️ Liked by Sebastian Raschka, PhD

It's a great overview. Thank you for sharing.

I really hope you'll publish another one in about six months.

♡ LIKE (1)    💬 REPLY



**1 reply by Sebastian Raschka, PhD**

---

© 2026 Raschka AI Research (RAIR) Lab LLC · [Privacy](#) · [Terms](#) · [Collection notice](#)  
[Substack](#) is the home for great culture

♡ LIKE (1)    💬 REPLY



**1 more comment...**